

Vision Meets Language

Building a Multimodal Transformer from Scratch

Seasons of Code · IIT Bombay

Mentee Handbook · Tasks 0, 1, 2

Coders Together Strong

Preface — How to Use This Document

This handbook covers the first three tasks of the project. It is the same document for every mentee, regardless of background. There are no separate tracks. If you are already comfortable with PyTorch and transformers, the curiosity boxes scattered throughout are where you should be spending your extra time. If you are newer to deep learning, focus on getting the core tasks working first, and treat the boxes as previews of things you will learn later.

Philosophy. We are building everything from scratch. That means no high-level wrappers, no `nn.MultiheadAttention`, no `nn.TransformerEncoderLayer`. The point is not to ship a model fast. The point is for every line of code you write to correspond to a piece of math you can explain to someone else. If you can't explain why your code does what it does, the code is not finished, even if it runs.

Writeups matter more than code. Code can be copied. Understanding cannot. Every task has a writeup component where you answer conceptual questions in your own words. And even if you use AI for writeups be sure to understand it. Be honest when you don't understand something — saying "I don't get why X" is much more valuable to your learning than pretending you do.

Time budgets are guidelines. Each task lists an expected number of hours. If you are spending dramatically more, that is a signal to ask for help, not a sign of weakness. Debugging shape mismatches and silent NaN bugs is part of the learning, but you should not lose three days to a bug you could have asked about in five minutes.

Curiosity boxes. Throughout the document you will see boxes labeled *Curiosity Corner*. These contain pointers to advanced material that is *not* required for the task but extends what you just learned. Examples include modern optimizer variants, advanced attention techniques, and tricks used in real-world systems. They exist so that mentees who finish early have somewhere interesting to go, and so that everyone leaves this project with a map of the landscape, even if they have only walked through part of it.

Prerequisites you should refresh before Task 0: basic PyTorch (tensors, autograd, simple training loop), linear algebra (matrix multiplication, dot product, transpose), and the chain rule. If any of these feel shaky, the first day of Task 0 is the time to catch up.

Expected Timeline

The full project spans roughly nine weeks. Below is the high-level arc. This document covers Tasks 0 through 2; subsequent tasks will be released as you progress.

Task	Duration	Theme
Task 0	2 days	Environment setup, tensor warmup, paper reading. Diagnostic and on-ramp.
Task 1	3 days	Bigram language model, then a single self-attention head from scratch.
Task 2	4 days	Multi-head attention, full transformer decoder, derivation of attention gradients.
Task 3	~1 week	Vision Transformer (ViT) — patching, position embeddings, image classification.
Task 4	~1 week	Cross-attention and the multimodal bridge — joining vision and language.
Task 5	~1 week	Contrastive learning, InfoNCE loss, training loop for paired data.
Task 6	~1 week	Training on Flickr8k or similar, debugging gradient flow, sampling.
Task 7	~1 week	Inference: text-to-image and image-to-text retrieval. Evaluation metrics.
Task 8	~1 week	Documentation, ablations, final writeup, presentation.

Tasks are released on completion, not on a fixed schedule. If the cohort is moving faster or slower, the timeline adjusts.

Task 0 — Setup, Tensor Warmup, and Paper Reading

Time budget: 6–8 hours over 2 days.

Goal: Get your environment working, prove to yourself that you can manipulate tensors fluently in PyTorch, and read your first ML research paper carefully.

Why this task exists

Every later task assumes you can write PyTorch code without fighting the basics. This is the task to discover gaps in those basics and fix them now, before they compound. The paper reading at the end is to build the muscle of extracting ideas from primary sources — almost everything you will implement in this project comes from a small set of papers, and you should be able to read them.

Part A — Environment Setup

Time: about 1 hour.

1. Install PyTorch. CPU-only is fine for Tasks 0–2. If you have access to a GPU, use it, but Google Colab's free GPU tier is sufficient for everything in this handbook.
2. Verify your install. Run the following and post the output in the group:

```
import torch
print(torch.__version__)
print(torch.cuda.is_available())
print(torch.tensor([1.0, 2.0]) @ torch.tensor([3.0, 4.0]))
```

3. Create a GitHub repository named SOC_multimodal_transformer-<roll_no.>. Make it public. Add a README with your name, a one-sentence intro, and a link to this handbook.
4. Create a folder structure: task0/, task1/, task2/, and so on. Each task's deliverables go in its own folder.

CURIOSITY CORNER — Beyond Python — what PyTorch actually runs

When you call `torch.matmul` on a tensor, you are not really executing Python. Underneath, PyTorch dispatches to highly optimized kernels written in C++ and CUDA. On CPU, it uses libraries like MKL (Intel) or OpenBLAS. On GPU, it uses cuBLAS and cuDNN. This is why a Python for-loop over tensor elements is hundreds of times slower than a single `matmul` — the `matmul` gets you out of Python and into a tight C loop.

Part B — Tensor Warmup

Time: 3–4 hours.

Solve the following in a single notebook, `task0/tensors.ipynb`. No `nn.Module`, no library shortcuts. The goal is to make sure every tool you'll use in the rest of the project is in your hands.

Problem 1 — Masked mean

Given x of shape (B, T, C) and y of shape (B, T) , compute the mean of x across the T dimension, but only at positions where $y == 1$. Output shape: (B, C) . No Python loops over batch or time. Use broadcasting and masking.

Problem 2 — Softmax from scratch

Implement softmax using only `exp`, `sum`, and arithmetic. Verify it matches `torch.softmax` to $1e-6$. Then explain in writing: why is naive softmax numerically unstable, and what is the standard trick to fix it?

CURIOSITY CORNER — Numerical stability and the log-sum-exp trick

If your logits include a value like 1000, then $\exp(1000)$ overflows to infinity and the whole softmax becomes NaN. The fix is to subtract the maximum logit before exponentiating: $\text{softmax}(x) = \text{softmax}(x - \max(x))$. This doesn't change the result mathematically — it cancels out in the ratio — but it keeps the largest exponent at $\exp(0) = 1$, so nothing overflows.

The same trick appears in log-probabilities under the name log-sum-exp. You will see it again in InfoNCE loss later in this project. Searching the term "log-sum-exp trick" is a worthwhile detour.

Problem 3 — Attention scores two ways

Given Q and K of shape (B, T, d) , compute the attention scores matrix (B, T, T) two ways: once with `torch.einsum` and once with `@` (matmul) plus transpose. Verify they are exactly equal.

CURIOSITY CORNER — Einsum is your friend

`torch.einsum` lets you write tensor contractions as readable strings. For attention scores: `einsum('btd,bsd->bts', Q, K)`. The letters name the axes; repeated letters get summed over. Once you internalize it, `einsum` makes high-dimensional tensor manipulations dramatically clearer than juggling transpose, `unsqueeze`, and `matmul`. The same notation works in NumPy and JAX. Highly recommended detour: Tim Rocktäschel's tutorial "Einsum is all you need".

Problem 4 — Causal mask

Build a causal mask of shape (T, T) that is 0 on and below the diagonal and $-\infty$ above. Apply it (by addition) to an attention score matrix before softmax. Visualize the resulting post-softmax attention matrix for $T=8$ as a heatmap.

Problem 5 — LayerNorm from scratch

Given x of shape (B, T, C) , implement LayerNorm: compute the mean and variance over the C dimension only (per-token, per-batch), normalize, then apply a learnable scale γ and shift β . Match `nn.LayerNorm` output to $1e-5$.

CURIOSITY CORNER — LayerNorm vs BatchNorm vs RMSNorm

BatchNorm normalizes across the batch dimension and works beautifully for CNNs on images, but fails for variable-length sequences and small batches. LayerNorm normalizes across the feature dimension per example, so it doesn't care about batch size — which is why every transformer uses it.

RMSNorm goes further: it drops the mean-centering step and only normalizes by root-mean-square. It is slightly cheaper and works just as well in practice. LLaMA and many modern LLMs use RMSNorm instead of LayerNorm.

Problem 6 — Manual gradients

Take `x = torch.randn(4, requires_grad=True)`. Compute `y = (x**2).sum()`, call `y.backward()`, and manually compute on paper what `x.grad` should be. Verify. Then repeat for `y = torch.softmax(x, dim=0).sum()` and explain why the gradient looks the way it does (hint: derivative of softmax has a specific structure).

CURIOSITY CORNER — Autograd is not magic

PyTorch's autograd builds a directed graph of operations as you run forward, then traverses it in reverse to apply the chain rule. Every operation knows how to compute its local gradient. When you write a custom layer, the autograd system handles backprop for you as long as you only use differentiable ops. If you ever need a custom backward — for non-differentiable steps like sampling, or for a more efficient gradient than the default — you subclass `torch.autograd.Function`. You will not need this in the project, but knowing it exists is useful.

Part C — Paper Reading

Time: 2–3 hours.

Read "Attention Is All You Need" by Vaswani et al., 2017. Sections 1 through 3.2 are required; the rest is optional for now. Then write a one-page response in `task0/paper_response.md` answering:

1. What problem were the authors solving, and what was the alternative they were replacing? (One paragraph.)
2. In your own words, what is self-attention computing? Don't paraphrase the paper's sentences — explain it as if to a friend who knows linear algebra but not deep learning.

3. List exactly three things from the paper you didn't fully understand. Be specific ("I don't get the multi-head concatenation" is good; "the math is hard" is not useful).

⚠ Don't skip the honest "didn't understand" list

This is the most important deliverable in Task 0. We are not testing whether you grasped the paper on first read — almost nobody does. We are testing whether you can identify your own gaps. Mentees who write "everything was clear" are usually the ones who struggle most in Task 1.

Deliverables

- `task0/tensors.ipynb` with all six problems solved and outputs visible.
- `task0/paper_response.md`
- Verify your install output posted in the group chat.
- Push everything to your GitHub repo and share the link.

Task 1 — Bigram Model and Your First Self-Attention Head

Time budget: 10–12 hours over 3 days.

Goal: Build the simplest possible language model, train it on Shakespeare, then bolt on a single self-attention head — written from scratch — and watch the loss drop.

Why this task exists

Self-attention is the single most important idea in this project. Before you build anything multi-head, before you stack blocks, before you touch ViT or cross-attention — you need to have built one attention head with your own hands and stared at what each line is doing. Everything else is a generalization of this one piece.

We anchor the task to language modeling on Shakespeare because it is small, the loss is easy to interpret (cross-entropy on characters), and the qualitative output is fun to look at.

Dataset

Tiny Shakespeare — a single text file, around 1 MB, containing the complete works of Shakespeare. Download from Andrej Karpathy's char-rnn repository on GitHub. Save as `task1/input.txt`.

Part A — Bigram Baseline

Time: about 3 hours.

Build a character-level bigram language model in a single file, `task1/bigram.py`. A bigram model predicts the next character based only on the current character — no context. The model is literally a lookup table of shape `(vocab_size, vocab_size)`.

Step by step:

4. Read the text file. Build a sorted list of all unique characters; this is your vocabulary.
5. Build the tokenizer: two dictionaries, `stoi` (string to int) and `itos` (int to string).
6. Encode the entire text into a tensor of integer token IDs.
7. Split 90/10 into train and validation tensors.
8. Write `get_batch(split)`: sample `B` random starting positions, take chunks of length `T+1`, and return `x` and `y` where `y` is `x` shifted by one.
9. Build a `BigramLanguageModel` using `nn.Embedding(vocab_size, vocab_size)`. The forward pass returns logits and (if targets are given) cross-entropy loss.
10. Training loop with AdamW, learning rate `1e-2`, 3000 steps, batch size 32, block size 8.

11. Implement `generate(idx, max_new_tokens)` using categorical sampling from the softmax of logits.

Expected outcome: training loss starts around 4.6 ($\approx \log(\text{vocab_size})$) and drops to roughly 2.5. Generated text is gibberish but has reasonable letter frequencies — lots of vowels, occasional spaces.

CURIOSITY CORNER — Why is the starting loss $\log(\text{vocab_size})$?

At initialization, your model knows nothing, so each of the `vocab_size` outputs is roughly equally likely. The probability of the correct token is therefore about $1/\text{vocab_size}$, and the cross-entropy loss is $-\log(1/\text{vocab_size}) = \log(\text{vocab_size})$. For tiny Shakespeare's 65 characters, that's about 4.17. This is a useful sanity check — if your starting loss is wildly different, your initialization or your loss computation is wrong.

CURIOSITY CORNER — AdamW versus SGD versus everything else

SGD with momentum is the classical optimizer — simple, robust, well-understood. It is still the workhorse for image classification with CNNs. Adam adds per-parameter adaptive learning rates by tracking running averages of gradients and their squares. AdamW corrects a subtle bug in Adam's interaction with weight decay (decoupling decay from the gradient update). For transformers, AdamW is the default for good reason: it handles the heterogeneous gradient magnitudes across embedding, attention, and MLP parameters gracefully.

Worth exploring later: Lion (Google, 2023) — uses only sign of gradient and is more memory-efficient. Sophia (Stanford, 2023) — uses a diagonal Hessian estimate for faster convergence on LLMs. Shampoo and its variants — full second-order methods that are gaining traction at scale. For this project, stick with AdamW.

Part B — Adding a Single Self-Attention Head

Time: 5–6 hours.

Now you will write attention yourself. Copy `bigram.py` to `task1/attention.py` and modify.

Conceptual setup. A self-attention head takes in a sequence of token embeddings and produces a sequence of "context-aware" embeddings. For each position t , it looks at all previous positions (because of the causal mask), decides which ones are relevant, and produces a weighted sum of their values. The mechanism by which it decides relevance is a learned similarity between queries (from position t) and keys (from all earlier positions).

Build the Head class:

```
class Head(nn.Module):
    def __init__(self, n_embd, head_size, block_size):
```

```

    super().__init__()
    self.key = nn.Linear(n_embd, head_size, bias=False)
    self.query = nn.Linear(n_embd, head_size, bias=False)
    self.value = nn.Linear(n_embd, head_size, bias=False)
    self.register_buffer(
        "tril",
        torch.tril(torch.ones(block_size, block_size))
    )

    def forward(self, x):
        # x: (B, T, C)
        B, T, C = x.shape
        k = self.key(x)      # (B, T, head_size)
        q = self.query(x)    # (B, T, head_size)
        v = self.value(x)    # (B, T, head_size)

        # attention scores
        wei = q @ k.transpose(-2, -1)           # (B, T, T)
        wei = wei * (k.size(-1) ** -0.5)        # scale by 1/sqrt(d_k)
        wei = wei.masked_fill(
            self.tril[:T, :T] == 0, float("-inf"))
        )
        wei = F.softmax(wei, dim=-1)            # (B, T, T)

        # weighted sum of values
        out = wei @ v                      # (B, T, head_size)
        return out

```

Integrate into the model. Replace the direct embedding-to-logits path with:

- Token embedding table.
- Position embedding table — also `nn.Embedding`, indexed by position 0 through T-1.
- Sum the two embeddings to get `x` of shape `(B, T, n_embd)`.
- Pass through one Head.
- Final `nn.Linear(head_size, vocab_size)` to get logits.

Hyperparameters: `n_embd=32`, `head_size=32`, `block_size=8`, `batch_size=32`, learning rate `1e-3`, 5000 steps.

Expected outcome: loss drops to around 2.4. Generated text is still mostly gibberish, but you should see slightly more word-like patterns.

⚠ **Don't expect huge gains from one head**

One attention head on tiny Shakespeare with only 8 tokens of context cannot do much. The loss only drops marginally from the bigram baseline. The point of this part is to see attention working at all, not to produce a good model. Save the celebration for Task 2.

CURIOSITY CORNER — Why scale by $\sqrt{d_k}$?

If Q and K are drawn from standard normal distributions and you take their dot product over d_k dimensions, the variance of the result grows linearly with d_k . With large d_k , the resulting attention scores have huge magnitudes. After softmax, this produces extremely peaked distributions where one position gets almost all the weight and everything else is essentially zero. Gradients through that softmax become tiny.

Dividing by $\sqrt{d_k}$ keeps the variance of the dot product roughly constant regardless of dimension, so softmax behaves well. This is one of the subtler details in the original Transformer paper — and one of the most asked-about. You'll prove this rigorously in Task 2.

Part C — Conceptual Writeup

Time: 2–3 hours.

Answer the following in `task1/writeup.md`. Write in prose, in your own words.

12. Why do we divide attention scores by $\sqrt{d_k}$? Connect your answer to the variance of dot products and to the shape of the softmax.
13. The causal mask is applied before softmax, by setting future-position scores to negative infinity. What would happen if instead you applied a mask after softmax by zeroing out those entries? Why is the before-softmax approach correct?
14. Describe Q , K , and V in your own words. An analogy is fine — but also give the linear-algebra view. What is each linear projection doing?
15. Your single-head attention model only marginally outperforms the bigram. Why? What is the bottleneck — capacity, context length, depth, or something else?
16. Paste 200 characters of generated text from the bigram model and 200 from the attention model. Describe the qualitative difference.

Deliverables

- `task1/bigram.py`
- `task1/attention.py`
- `task1/samples.txt` with 200-character samples from both models.

- `task1/writeup.md`

Task 2 — Multi-Head Attention and a Full Transformer Decoder

Time budget: 14–16 hours over 4 days.

Goal: Generalize one head to many. Build the full transformer block (attention + MLP + LayerNorm + residuals), stack four of them, train, and derive the attention gradient by hand.

Why this task exists

By the end of this task you will have built, with no shortcuts, a working decoder-only transformer — the same architectural family as GPT-2, GPT-3, and most modern LLMs. Every component you implement here returns in Task 3 (ViT is the same building block with a different input) and in Task 4 (cross-attention is a small variation on what you build now). This is the architectural foundation of the entire project.

Part A — Multi-Head Attention

Time: 3–4 hours.

A single head learns one set of (Q, K, V) projections — one way of looking at the sequence. Multi-head attention runs several heads in parallel, each with its own learned projections, then concatenates their outputs and projects back to the model dimension. Different heads can specialize: one might track syntactic agreement, another might attend to the immediately preceding token, another might find semantic relationships.

Implementation:

```
class MultiHeadAttention(nn.Module):
    def __init__(self, n_embd, n_head, head_size, block_size):
        super().__init__()
        self.heads = nn.ModuleList([
            Head(n_embd, head_size, block_size)
            for _ in range(n_head)
        ])
        self.proj = nn.Linear(n_head * head_size, n_embd)
        self.dropout = nn.Dropout(0.2)

    def forward(self, x):
        out = torch.cat([h(x) for h in self.heads], dim=-1)
        out = self.dropout(self.proj(out))
        return out
```

Hyperparameters from here on: $n_embd=128$, $n_head=4$, $head_size=32$ (so total output is 128, matching n_embd).

CURIOSITY CORNER — The for-loop is not how production code does it

Concatenating outputs from a Python for-loop over heads is correct but slow. In real implementations, the heads are computed as a single batched matrix multiplication by reshaping (B, T, n_embd) into (B, T, n_head, head_size) and treating n_head as a batch dimension. This gives you 4x speedup for free.

Beyond that, FlashAttention (Tri Dao et al., 2022) reorganizes attention computation to keep things in fast GPU SRAM rather than slow HBM. It is mathematically identical but 2–4x faster and uses much less memory. Modern PyTorch ships with `scaled_dot_product_attention` which uses FlashAttention under the hood when applicable.

CURIOSITY CORNER — Dropout — what and why

Dropout randomly zeros a fraction of activations during training (and scales the rest up to keep the expected sum constant). At inference it is disabled. The original motivation, from Hinton et al., was that it prevents co-adaptation between units, forcing the network to learn redundant, robust representations.

A modern view is that dropout is a form of stochastic regularization that approximates training an ensemble of subnetworks. In transformers it is applied after attention, after MLPs, and on residual paths. Typical rates: 0.1 for large models, 0.2 for small ones like ours.

Part B — The Transformer Block

Time: 3–4 hours.

A transformer block has four pieces wrapped together: multi-head attention, a feed-forward MLP, two LayerNorms, and two residual connections. The pattern is:

```
x = x + self.attn(self.ln1(x))
x = x + self.ffn(self.ln2(x))
```

This is *pre-norm* — LayerNorm applied *before* each sub-layer. The original "Attention Is All You Need" paper used post-norm (LayerNorm after each sub-layer). Pre-norm trains more reliably for deep networks because the residual path stays clean; most modern transformers use pre-norm.

The MLP / feed-forward block:

```
class FeedForward(nn.Module):
    def __init__(self, n_embd):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(n_embd, 4 * n_embd),
            nn.ReLU(),
            nn.Linear(4 * n_embd, n_embd),
```

```

        nn.Dropout(0.2),
    )
    def forward(self, x):
        return self.net(x)

```

Why 4x? The expansion factor of 4 is convention from the original Transformer paper and has stuck because it consistently works well. The MLP is where most of the parameters live — they are wider than they are deep.

CURIOSITY CORNER — ReLU is fine; modern transformers use GELU or SwiGLU

$\text{ReLU}(x) = \max(0, x)$. Simple, fast, and works. But GELU (Gaussian Error Linear Unit) — used in BERT, GPT-2, and many others — gives slightly better performance because it has a smooth, non-monotonic shape around zero. Even better, SwiGLU (Swish + Gated Linear Unit, from Shazeer 2020) is used in LLaMA and other modern LLMs; it doubles the MLP parameters but gives meaningfully better quality per parameter spent.

For this project, stick with ReLU. But know what's out there.

Now build the Block:

```

class Block(nn.Module):
    def __init__(self, n_embd, n_head, block_size):
        super().__init__()
        head_size = n_embd // n_head
        self.attn = MultiHeadAttention(n_embd, n_head, head_size, block_size)
        self.ffn = FeedForward(n_embd)
        self.ln1 = nn.LayerNorm(n_embd)
        self.ln2 = nn.LayerNorm(n_embd)

    def forward(self, x):
        x = x + self.attn(self.ln1(x))
        x = x + self.ffn(self.ln2(x))
        return x

```

Full model:

- Token embedding + position embedding (still summed).
- Four Block modules stacked.
- Final `nn.LayerNorm`.
- Final `nn.Linear(n_embd, vocab_size)` to logits.

CURIOSITY CORNER — Why residual connections matter so much

Residuals ($x + \text{sublayer}(x)$) serve two purposes. First, they give gradients a highway: the gradient of x w.r.t. x is identity, so the backward signal reaches early layers without vanishing through many multiplications. Without them, training deep transformers is nearly impossible. Second, they bias

the network toward learning identity-plus-small-correction, which is often easier to optimize than learning a transformation from scratch.

You will demonstrate exactly how catastrophic their removal is in Part D.

Training hyperparameters: $B=64$, $T=64$, $n_embd=128$, $n_head=4$, $n_layer=4$, $lr=3e-4$, $dropout=0.2$, 5000 steps. On CPU this takes ~30 minutes; on a free Colab T4 GPU, about 5 minutes.

Expected outcome: loss drops to roughly 1.5. Generated text is still nonsensical, but it has the shape of English — capitalization after periods, recognizable word lengths, occasional plausible word fragments.

CURIOSITY CORNER — Learning rate scheduling — the warmup-cosine recipe

For this task, a constant learning rate is fine. But every serious transformer training run uses a schedule. The standard recipe: linear warmup for the first ~1000 steps (from 0 up to the peak LR), then cosine decay down to 10% of peak over the rest of training. The warmup prevents Adam's adaptive learning rates from diverging while its running statistics are still noisy. The cosine decay gives a clean smooth annealing.

Search terms: "transformer learning rate warmup", "cosine schedule", "OneCycle LR".

Part C — Derive Attention Gradients by Hand

Time: 3–4 hours.

This is the part of the task most people will be tempted to skip. Do not skip it. Submit as `task2/math.pdf` — handwritten on paper, photographed, and converted to PDF is perfectly fine. Clean handwriting beats messy LaTeX.

Setup. Let:

- $S = Q K^T / \sqrt{d_k}$ (the unscaled-then-scaled scores, shape $T \times T$)
- $P = \text{softmax}(S)$ (row-wise softmax, shape $T \times T$, each row sums to 1)
- $A = P V$ (the attention output, shape $T \times d_v$)

Derivations to complete:

17. **1. Warmup.** Derive $\partial A / \partial V$. This is straightforward — A is linear in V given P . Express the answer in terms of P .
18. **2. Softmax Jacobian.** Show that for $p = \text{softmax}(s)$, the Jacobian $\partial p / \partial s$ has entries: $\partial p_i / \partial s_j = p_i (\delta_{ij} - p_j)$, where δ_{ij} is the Kronecker delta. Write this out clearly.
19. **3. Main result.** Derive $\partial A / \partial Q$. Use the chain rule: $\partial A / \partial Q = (\partial A / \partial P)(\partial P / \partial S)(\partial S / \partial Q)$. Show each step. Keep the indices straight.

20. **4. Interpretation.** In one paragraph: explain why the gradient through softmax becomes very small when the input logits have large magnitudes. Connect this directly back to the $\sqrt{d_k}$ scaling factor. This is the formal version of what you wrote in Task 1.

 This is the single most important exercise in Tasks 0–2

Almost nobody who calls themselves a transformer expert has actually derived this. If you do it once, by hand, you will understand attention at a level that survives every future paper, every architecture variant, every "what if we changed X" question. Take your time. Get stuck. Look things up. The struggle is the point.

 CURIOSITY CORNER — Beyond first-order: where Hessians and second-order methods come in

Backpropagation gives you first-order gradients — enough to do gradient descent. But the curvature of the loss landscape is captured by the Hessian (the matrix of second derivatives). Computing the full Hessian for a transformer is hopelessly expensive — its size is parameters squared. But approximations of it power some advanced optimizers like K-FAC, Shampoo, and Sophia.

More immediately relevant: when you read about "gradient clipping", "learning rate as a substitute for inverse Hessian", or "why Adam works", these are all about navigating loss curvature with only first-order information.

Part D — Ablation Experiments

Time: 2–3 hours.

Run three variants of your 4-block model. Plot all three training-loss curves on the same axes.

21. **Variant 1 — Baseline.** As described above. 5000 steps.
22. **Variant 2 — No residual connections.** Inside the Block forward, change to `x = self.attn(self.ln1(x))` (no `x +`). Same for the MLP. Train 5000 steps. What happens to the loss?
23. **Variant 3 — No LayerNorm.** Remove both LayerNorms from the Block (keep residuals). Train 5000 steps. What happens?

Write one paragraph per variant describing what you observed, then explain why in terms of gradient flow.

 CURIOSITY CORNER — Ablation studies — the workhorse of ML research

"Does feature X actually matter?" is answered by an ablation: remove X, retrain, compare. Almost every ML paper has an ablation section. Doing them yourself is the fastest way to develop intuition about which architectural choices matter and which are decoration. When you read papers, the ablation table is often more informative than the main results table.

Part E — Final Writeup

Time: 1–2 hours.

In `task2/writeup.md`:

24. What is the role of the MLP in each block? Attention mixes information across positions. What does the MLP do that attention cannot?
25. Pre-norm versus post-norm: which did you use, and why is pre-norm easier to train for deep networks?
26. Paste 300 characters of generated text. Compare qualitatively to Task 1's output.
27. From your ablation: which was more catastrophic, removing residuals or removing LayerNorm? Explain in terms of gradient flow what each removal breaks.
28. What did you find hardest in this task? What clicked unexpectedly?

Deliverables

- `task2/transformer.py`
- `task2/math.pdf` with all four derivations.
- `task2/ablation_plot.png` with three loss curves on one chart.
- `task2/samples.txt` with 300-character samples from the trained model.
- `task2/writeup.md`

What Comes After Task 2

Once Task 2 is solid, you will have, in your own hands, a working decoder-only transformer. The path forward becomes much clearer:

- **Task 3 — Vision Transformer (ViT).** The same attention you just built, but applied to image patches instead of text tokens. The only architectural new idea is patch embedding.
- **Task 4 — Cross-attention.** A small generalization where queries come from one modality and keys/values come from another. This is the bridge between vision and language.
- **Task 5 — Contrastive learning.** Train the model to align image and text representations using InfoNCE loss.
- **Tasks 6–8.** Train on real paired data, evaluate retrieval, write things up.

Every one of these builds directly on what you've done in Tasks 0–2. If you got Task 2 working and the writeup feels honest, you're ready.

Good luck. Have fun. Ask questions. The bug you're stuck on is probably one a teammate just unblocked themselves on — ask in the group. Coders together strong.